10강. 리덕스 툴킷(Redux Toolkit)

React

리덕스(Redux)

- **Redux란?**

  애플리케이션의 **전역 상태(state)를 관리하기 위한 상태 관리 라이브러리**입니다. 규모가 커질수록 복잡해지는 상태 관리를 예측 가능하게 만들어주는 것이 핵심입니다.

- **기존 리엑트 애플리케이션의 단점**
  - 탑-다운 방식을 사용하므로 부모-자식 사이에서만 데이터를 전달할 수 있다.
  - 자식 컴포넌트 사이에서는 직접적인 데이터 전달이 불가능하다.
  - 컴포넌트가 많아질수록 상태 관리는 복잡하고 어려워진다.

리덕스(Redux)

- **Redux와 useReuducer의 차이**

구분	useReducer	Redux
범위	컴포넌트 내부	전역
설치	필요 없음	필요 (`redux`, `react-redux`)
복잡도	낮음	높음
상태 공유	어렵다 (props/context 필요)	매우 쉬움
사용 목적	로컬 상태 관리	앱 전체 상태 관리
구조	간단	구조화된 아키텍처

리덕스(Redux)

- **Redux란?**

1) Store (저장소)

- 애플리케이션의 **모든 상태를 저장**

```
const store = createStore(reducer);
```

2) Action (행동)

- 상태를 변경하기 위한 **요청 객체**

```
{
  type: "INCREMENT"
}
```

리덕스(Redux)

- **Redux란?**

3) Reducer (상태 변경 함수)

- 이전 상태 + action → 새로운 상태 반환

```
function counterReducer(state = { count: 0 }, action)
{
  switch (action.type) {
    case "INCREMENT":
      return { count: state.count + 1 };
    case "DECREMENT":
      return { count: state.count - 1 };
    default:
      return state;
  }
}
```

리덕스 툴킷(Redux Toolkit)

- **Redux의 단점**

  액션 타입정의, 액션 생성자, 리듀서 작성 등 많은 설정과 반복적인 코드가 필요하다.

- **Redux 툴킷(Tool-Kit)**

  Redux Toolkit(RTK)은 **Redux를 더 쉽고 간결하게 사용하기 위한 공식 라이브러리**입니다.

  리덕스의 핵심 개념은 유지하면서도 createSlice, configureStore 같은 API를 제공하여 복잡한 설정을 줄이고, 간결하면서도 효율적으로 상태관리를 구현할 수 있도록 도와줍니다.

리덕스 툴킷(Redux Toolkit)

- **React Redux 공식 사이트**

React Redux

Getting Started Tutoria

Introduction
- Getting Started
- Why Use React Redux?

Tutorials
- Quick Start
- TypeScript Quick Start
- Tutorial: Connect API

Using React Redux
- Usage with TypeScript
- Connect: Extracting Data with mapStateToProps
- Connect: Dispatching Actions with mapDispatchToProps
- Accessing the Store

API Reference

Guides

Usage Summary

Install Redux Toolkit and React Redux

Add the Redux Toolkit and React Redux packages to your project:

```
npm install @reduxjs/toolkit react-redux
```

Create a Redux Store

Create a file named `src/app/store.js`. Import the `configureStore` A
creating an empty Redux store, and exporting it:

app/store.js

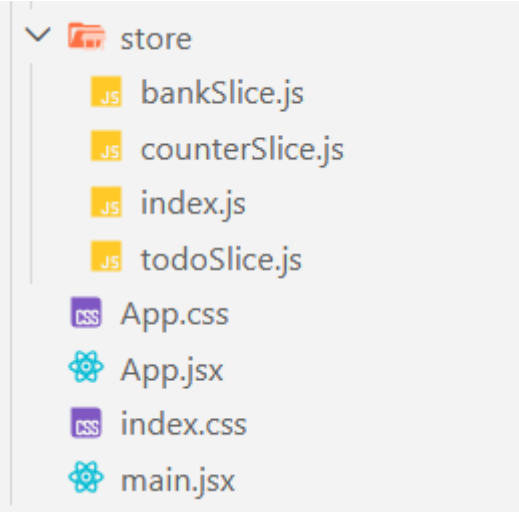
```
import { configureStore } from '@reduxjs/toolkit'

export default configureStore({
  reducer: {},
})
```

리덕스 툴킷(Redux Toolkit)

- **Redux 툴킷(Tool-Kit) 설치**

```
npm install @reduxjs/toolkit react-redux
```

```
"dependencies": {
  "@reduxjs/toolkit": "^2.11.2",
  "react": "^19.2.4",
  "react-dom": "^19.2.4",
  "react-redux": "^9.2.0",
  "react-router-dom": "^7.14.0"
},
```



리덕스 툴킷(Redux Toolkit)

- **createSlice** - action + reducer를 한 번에 생성

```
import { createSlice } from '@reduxjs/toolkit'
const initialState = {
  count: 0 // 카운트의 초기값을 0으로 설정
}

const counterSlice = createSlice({
  name: 'counter', // slice의 이름을 지정
  initialState,
  reducers: {
    // 카운트 값을 1 증가시키는 reducer
    increment: (state) => {
      state.count += 1
    },
  }
})
// 액션 생성자와 리듀서를 export
export const { increment} = counterSlice.actions
export default counterSlice.reducer
```

counterSlice.js

리덕스 툴킷(Redux Toolkit)

- **store 생성 – configureStore()**

reducer를 등록

index.js

```
import { configureStore } from '@reduxjs/toolkit'
import counterReducer from './counterSlice'


export const store = configureStore({
  reducer: {
    // counterSlice에서 export한 reducer를 등록
    counter: counterReducer,
  }
})
```

리덕스 툴킷(Redux Toolkit)

- **Provider**

  Redux의 store를 React와 연결하기 위해 사용

main.jsx

```
import { Provider } from 'react-redux'
import { store } from './store/index.js'


createRoot(document.getElementById('root')).render(
    <Provider store={store}>
      <App />
    </Provider>
)
```

리덕스 툴킷(Redux Toolkit)

- **useSelector**

store의 state를 가져오고, state가 변할 때마다 업데이트 함

```
import { useSelector, useDispatch } from "react-redux"
// counterSlice에서 export한 액션 생성 함수를 가져옴
```

apps/counter.jsx

```
const Counter = () => {
  // store의 state에서 count 값을 가져옴
  const count = useSelector(state => state.counter.count)
```

리덕스 툴킷(Redux Toolkit)

- **useDispatch**

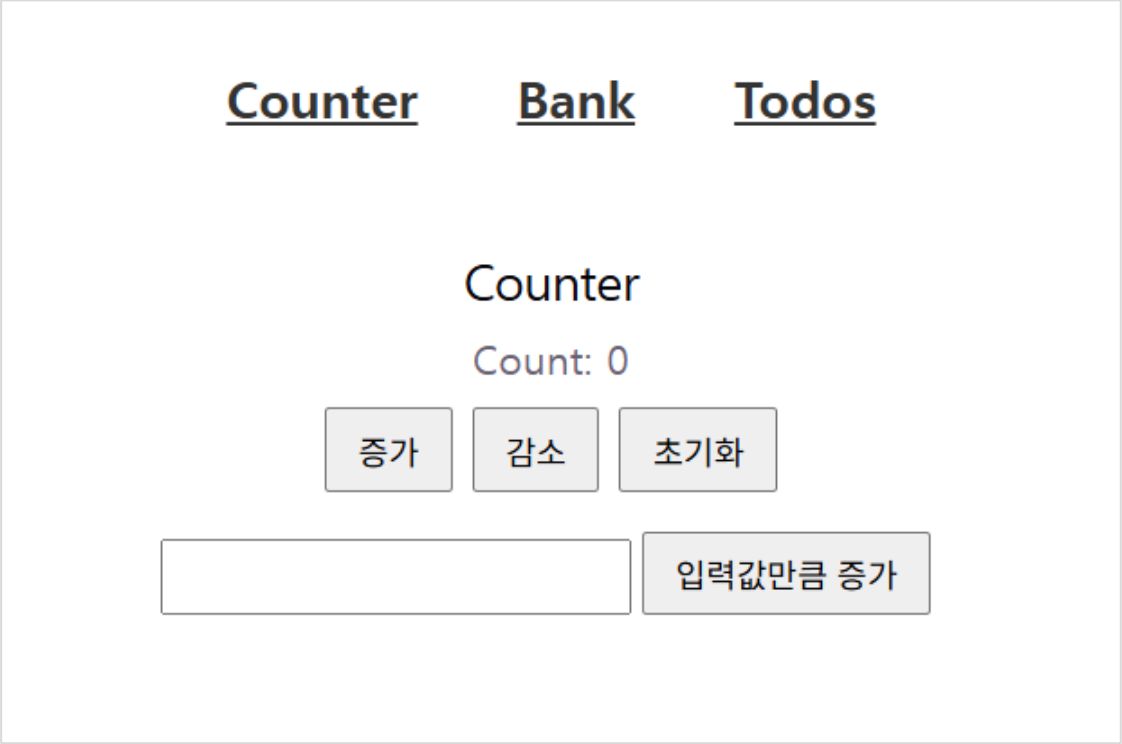
store의 dispatch 메서드를 반환, action을 dispatch 할 수 있음

```
// dispatch 함수를 가져옴 - 액션을 dispatch할 때 사용
  const dispatch = useDispatch()

  return (
    <div className="redux-app">
      <h2>Counter</h2>
      <div>
        <p>Count: {count}</p>
        <button onClick={() => dispatch(increment())}>증가</button>
```

apps/counter.jsx

Counter 앱

- **Counter**



Counter 앱

- **Counter**

```
function App() {

  return (
    <>
      <section id="center">
        <BrowserRouter>
          <Header />
          <Routes>
            <Route path="/" element={<Counter />} />
            <Route path="/counter" element={<Counter />} />
            <Route path="/bank" element={<Bank />} />
            <Route path="/todos" element={<Todos />} />
          </Routes>
        </BrowserRouter>
      </section>
    </>
  )
}
```

Counter 앱

- **Counter**

```
import { NavLink, Link } from "react-router-dom"

const Header = () => {
  return (
    <header>
      <nav>
        <NavLink to="/counter">Counter</NavLink>
        <NavLink to="/bank">Bank</NavLink>
        <NavLink to="/todos">Todos</NavLink>
      </nav>
    </header>
  )
}
```

Counter 앱

- **Counter**

```
import { configureStore } from '@reduxjs/toolkit'
import counterReducer from './counterSlice'
import bankReducer from './bankSlice'
import todoReducer from './todoSlice'

export const store = configureStore({
  reducer: {
    // counterSlice에서 export한 reducer를 등록
    counter: counterReducer,

    // bankSlice에서 export한 reducer를 등록
    bank: bankReducer,

    // todoSlice에서 export한 reducer를 등록
    todo: todoReducer
  }
})
```

Counter 앱

- **Counter**

```
import { createSlice } from '@reduxjs/toolkit'

const initialState = {
  count: 0 // 카운트의 초기값을 0으로 설정
}

const counterSlice = createSlice({
  name: 'counter', // slice의 이름을 지정
  initialState,
  reducers: {  // state를 변경하는 reducer 함수를 정의
    // 카운트 값을 1 증가시키는 reducer
    increment: (state) => {
      state.count += 1
    },
    // 카운트 값을 1 감소시키는 reducer
    decrement: (state) => {
      state.count -= 1
    },
```

Counter 앱

- **Counter**

```
    // 카운트 값을 action.payload로 증가시키는 reducer
    incrementByAmount: (state, action) => {
      state.count += action.payload
    },
    // 카운트 초기화
    reset: (state) => {
      state.count = 0
    }
  }
})

// 액션 생성자와 리듀서를 export
export const {
  increment,
  decrement,
  incrementByAmount,
  reset
} = counterSlice.actions
export default counterSlice.reducer
```

Counter 앱

- **Counter**

```
import {
  increment,
  decrement,
  incrementByAmount,
  reset
} from "../store/counterSlice"
import { useState } from "react"

import { useSelector, useDispatch } from "react-redux"
// counterSlice에서 export한 액션 생성 함수를 가져옴

const Counter = () => {
  // store의 state에서 count 값을 가져옴
  const count = useSelector(state => state.counter.count)
  // input에 입력된 값을 저장할 state
  const [inputValue, setInputValue] = useState(0)

  // input 값이 변경될 때마다 state를 업데이트하는 함수
  const handleInputChange = (e) => {
    setInputValue(Number(e.target.value))
  }
```

Counter 앱

- **Counter**

```
// dispatch 함수를 가져옴 - 액션을 dispatch할 때 사용
const dispatch = useDispatch()

// 입력된 값만큼 카운트를 증가시키는 함수
const handleIncrementAmount = () => {
  dispatch(incrementByAmount(inputValue))
}

return (
  <div className="redux-app">
    <h2>Counter</h2>
    <div>
      <p>Count: {count}</p>
      <button onClick={() => dispatch(increment())}>증가</button>
      <button onClick={() => dispatch(decrement())}>감소</button>
      <button onClick={() => dispatch(reset())}>초기화</button>
    </div>
```

Counter 앱

- **Counter**

```
        <div>
          <input
            type="number"
            onChange={handleInputChange}
          />
          <button onClick={handleIncrementAmount}>입력값만큼 증가</button>
        </div>
      </div>
    )
  }

  export default Counter
```

Bank System 앱

- **Bank**

Bank System 앱

- **Bank**

```
import { createSlice } from '@reduxjs/toolkit'

const initialState = {
  balance: 0, // 은행 잔액을 저장하는 state
  transactions: [] // 거래 내역을 저장하는 배열
}

const bankSlice = createSlice({
  name: 'bank',
  initialState,
  reducers: {
    deposit: (state, action) => {
      state.balance += action.payload  // 입금액을 잔액에 더함
      // 거래 내역에 입금 기록을 추가
      // push()를 사용하여 transactions 배열에 새로운 거래 객체를 추가
      state.transactions.push({
        type: 'deposit',
        amount: action.payload,
        // timestamp는 거래가 발생한 시간을 문자열로 저장
        timestamp: new Date().toLocaleString('ko-KR')
      })
    },
```

Bank System 앱

- **Bank**

```
    withdraw: (state, action) => {
      if (state.balance >= action.payload) {
        state.balance -= action.payload  // 출금액을 잔액에서 뺌
        // 거래 내역에 출금 기록을 추가
        state.transactions.push({
          type: 'withdraw',
          amount: action.payload,
          timestamp: new Date().toLocaleString('ko-KR')
        })

      } else {
        alert("잔액이 부족합니다.")
      }
    }
  }
})

export const { deposit, withdraw } = bankSlice.actions
export default bankSlice.reducer
```

Bank System 앱

- **Bank**

```
import { useSelector, useDispatch } from "react-redux"
import { deposit, withdraw } from "../store/bankSlice"
import { useState } from "react"

const ReduxBank = () => {
  // 입금/출금할 금액을 관리하는 state
  const [amount, setAmount] = useState(0)
  // bankSlice의 balance 값을 가져옴
  const balance = useSelector(state => state.bank.balance)
  // bankSlice의 transactions 값을 가져옴
  const transactions = useSelector(state => state.bank.transactions)

  const dispatch = useDispatch();  // dispatch 함수를 가져옴

  // 입금 버튼 클릭 시 deposit 액션을 dispatch
  const handleDeposit = () => {
    dispatch(deposit(Number(amount)))
  }

  // 출금 버튼 클릭 시 withdraw 액션을 dispatch
  const handleWithdraw = () => {
    dispatch(withdraw(Number(amount)))
  }
```

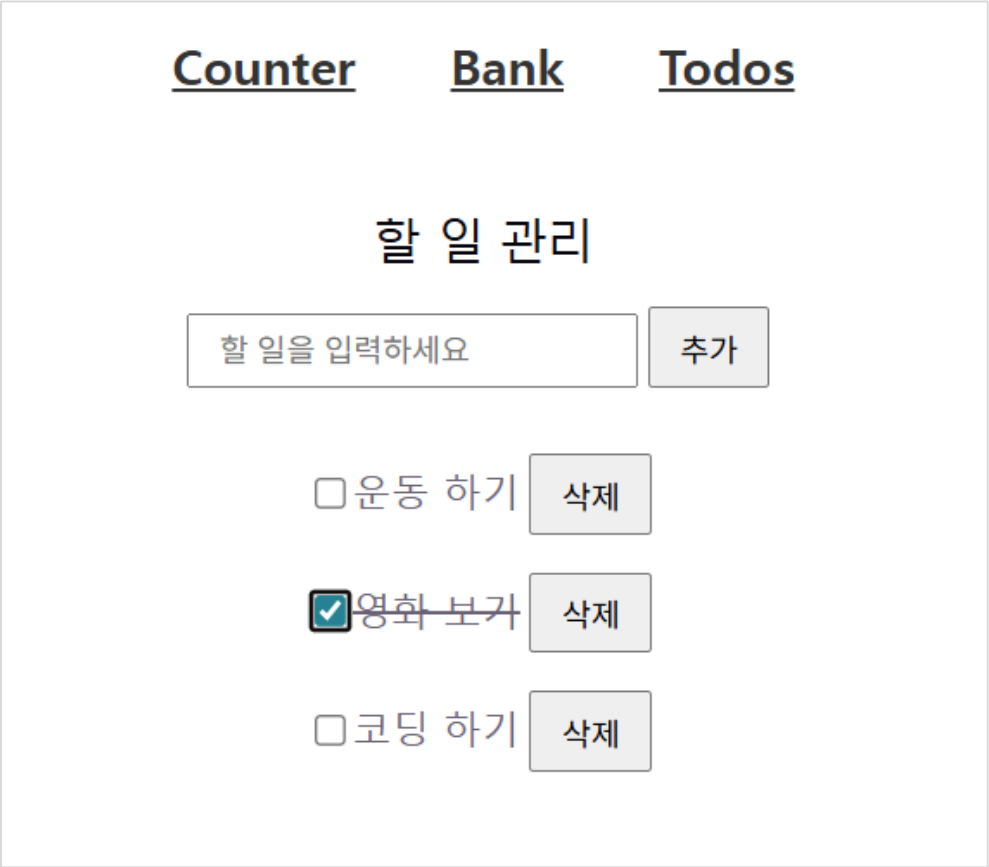
Bank System 앱

- **Bank**

```
return (
  <div>
    <h2>Bank System</h2>
    <h3>잔액: {balance}원</h3>
    <input
      type="number"
      value={amount}
      onChange={(e) => setAmount(e.target.value)}
      step="1000"
    />
    <button onClick={handleDeposit}>입금</button>
    <button onClick={handleWithdraw}>출금</button>
    <h3>거래 내역</h3>
    <ul>
      {transactions.map((transaction, index) => (
        <li key={index}>
          [{transaction.timestamp}]
            {transaction.type === 'deposit' ? '입금' : '출금'}: {transaction.amount}원
        </li>
      ))}
    </ul>
  </div>
)
```

To-do 앱

- **To-do app**



To-do 앱

- **To-do app**

```
const initialState = {
  todos: []
}

// createSlice 함수를 사용하여 todoSlice 생성
const todoSlice = createSlice({
  name: 'todo', // slice의 이름
  initialState,
  reducers: {
    // 새로운 할 일을 추가하는 reducer
    addTodo: (state, action) => {
      state.todos.push(action.payload)
    },
```

To-do 앱

- **To-do app**

```
    //할 일 체크 기능 추가
    toggleTodo: (state, action) => {
      const todo = state.todos.find(todo => todo.id === action.payload)
      if (todo) {
        todo.completed = !todo.completed
      }
    },
    // 특정 id를 가진 할 일을 제거하는 reducer
    removeTodo: (state, action) => {
      state.todos = state.todos.filter(todo => todo.id !== action.payload)
    }
  }
})

export const { addTodo, toggleTodo, removeTodo } = todoSlice.actions
export default todoSlice.reducer
```

To-do 앱

- **To-do app**

```
import { use, useState } from 'react'
import { useSelector, useDispatch } from 'react-redux'
import { addTodo, toggleTodo, removeTodo } from '../store/todoSlice'

const Todos = () => {
  const [inputValue, setInputValue] = useState('')
  const todos = useSelector(state => state.todo.todos)
  const dispatch = useDispatch()

  console.log(todos);

  // 할 일 입력 핸들러
  const handleInputChange = (e) => {
    // console.log(e.target.value);
    setInputValue(e.target.value)
  }
```

To-do 앱

- **To-do app**

```
// 할 일 추가 핸들러
const handleAddTodo = () => {
  if (inputValue.trim() !== '') {
    dispatch(addTodo({
      id: Date.now(), // 고유한 id 생성
      text: inputValue,
      completed: false
    }))
    setInputValue('')
  }
}

// 할 일 체크 기능 추가
const handleToggleTodo = (id) => {
  dispatch(toggleTodo(id))
}

// 할 일 삭제 기능 추가
const handleRemoveTodo = (id) => {
  dispatch(removeTodo(id))
}
```

To-do 앱

- **To-do app**

```
return (
  <div>
    <h2>할 일 관리</h2>
    <input
      type="text"
      placeholder="할 일을 입력하세요"
      value={inputValue}
      onChange={handleInputChange}
    />
    <button onClick={handleAddTodo}>추가</button>
    {/* 할 일 목록 */}
    <ul>
      {todos.map(todo => (
        <li key={todo.id} className={todo.completed ? 'completed' : ''}>
          <input
            type="checkbox"
            checked={todo.completed}
            onChange={() => handleToggleTodo(todo.id)}
          />
          {todo.text}
          <button onClick={() => handleRemoveTodo(todo.id)}>삭제</button>
        </li>
      ))}
```